

PROJECT REPORT OF CO-OP Project at Industry (Module-2)

ON

Code Sync

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



Submitted by: Jai Goel (2210991693)

Ishika Goel (2210991690)

Kavita (2210991774)

Supervised By: Dr.Preeti Saini

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

CONTENTS

S.No.	Title	Page No.
1.	Introduction	3
2.	Abstract	4
3.	Problem Statement	5-7
4.	Implementation	8-9
5.	Appendices	10-11
6.	Data Base Schema	12-19
7.	Entity Relationship Diagram	20-23
8.	Limitation and benefits	24-25
9.	Future Scope	26
10.	Conclusion	27
11.	References	28

DECLARATION

I hereby certify that the work which is being presented in the project report entitled **CodeSync** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Preeti Saini**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura
Date: 28 Apr, 2026

Jai Kumar Goel
2210991693

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Preeti Saini
Mentor
Department of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology,
Chitkara University, Punjab, India

ACKNOWLEDGEMENT

It is our pleasure to be indebted to various people, who directly or indirectly contributed in the development of this work and who influenced my thinking, behavior, and acts during the course of my study.

We express our sincere gratitude to all for providing me an opportunity to undergo Integrated Project as part of the curriculum.

We are thankful to **Dr. Preeti Saini** for his support, cooperation, and motivation provided to us during the training for constant inspiration, presence, and blessings.

Lastly, we would like to thank the almighty and our parents for their moral support and friends with whom we shared our day-to-day experience and received lots of suggestions that improve our quality of work.

1. Introduction

The advancement of digital technologies and the widespread adoption of remote work, online education, and globally distributed software development have transformed the way individuals and teams collaborate. Organizations, educational institutions, and development teams increasingly rely on virtual environments to communicate, share knowledge, and build software across different locations and time zones. However, despite the availability of numerous tools for coding, communication, and documentation, these tools often function as separate and disconnected systems. Developers and learners are required to switch between multiple platforms for writing code, conducting meetings, exchanging messages, and maintaining documentation. This constant context switching not only disrupts workflow continuity but also leads to decreased productivity, miscommunication, and inefficiencies in collaborative efforts.

To overcome these challenges, the **CodeSync** Project is designed as a comprehensive, all-in-one collaborative platform that brings together essential tools required for effective teamwork. The primary goal of **CodeSync** is to simplify and enhance collaboration for developers, educators, students, and remote teams by providing a unified, real-time working environment. The platform offers a web-based collaborative coding workspace that supports more than 80 programming languages, enabling multiple users to write, execute, and debug code simultaneously. This feature promotes interactive learning, pair programming, and efficient team-based development. In addition to coding capabilities, CodeSync integrates real-time communication tools such as video conferencing and instant messaging, allowing users to discuss ideas, resolve issues, and provide feedback without leaving the platform. Furthermore, a shared collaborative notepad supports documentation, brainstorming, and note-taking, ensuring that all project-related information is centrally accessible. By eliminating the need to rely on multiple separate applications, CodeSync aims to improve workflow efficiency, enhance communication, and create a seamless, productive, and engaging collaborative experience.

2. Abstract

With the rapid growth of remote work, online education, and globally distributed development teams, effective collaboration has become a critical requirement in modern software development. However, most existing collaboration solutions are fragmented, forcing users to rely on multiple disconnected tools for coding, communication, documentation, and version control. This results in increased context switching, reduced productivity, and inefficient teamwork.

The proposed system, CodeSync, aims to address these challenges by providing a unified, web-based collaborative development platform. The system enables real-time code editing where multiple users can work on the same code file simultaneously, ensuring instant synchronization using WebSocket technology. Integrated version control support allows users to manage code repositories efficiently through operations such as commit, push, pull, branching, and merging directly within the platform. Additionally, built-in code review and commenting features support structured feedback and collaborative problem-solving.

Developed using modern web technologies such as React.js for the frontend, Node.js and MongoDB for the backend, and Socket.io for real-time communication, the platform ensures scalability, responsiveness, and ease of use. By combining coding, communication, and collaboration into a single interface, CodeSync significantly improves development efficiency, reduces workflow fragmentation, and enhances team productivity in remote and distributed environments.

3. Problem Statement

In today's rapidly growing digital ecosystem, remote work, online education, and globally distributed development teams have become increasingly common. However, effective collaboration remains a major challenge due to the fragmented nature of existing tools. Developers and teams often rely on multiple disconnected platforms for coding, communication, documentation, and project management—such as separate code editors, video conferencing tools, chat applications, and note-taking systems.

This fragmentation leads to frequent context switching, reduced productivity, delayed feedback, and miscommunication among team members. Additionally, many existing solutions do not provide true real-time collaborative coding, have limited multi-language support, or fail to offer an integrated workspace that combines coding, communication, and collaboration into a single unified interface.

As a result, there is a strong need for a centralized collaboration platform that enables real-time code editing, seamless communication, efficient version control, and structured code reviews—thereby improving teamwork, productivity, and overall development efficiency.

3.1 Software and Hardware Requirement Specification

- **Methods**

1. Real-Time Code Editing
 - Allow multiple users to edit the same code file simultaneously.
 - Synchronize changes instantly using WebSockets or similar real-time communication technologies.
2. Version Control Integration
 - Integrate Git or implement a simplified version control system. ○ Enable users to perform operations such as commit, push, pull, branch, and merge directly within the platform.
3. Code Review and Comments
 - Allow users to add comments on specific lines of code.
 - Support threaded discussions, feedback, and improvement suggestions.

- **Programming / Working Environment Frontend:**

React.js Tailwind CSS

Backend /Database:

Node.js

MongoDB

Real-Time Communication:

WebSockets (Socket.io for Node.js or equivalent libraries)

Version Control: Git , GitHub

3.2 Requirements to Run the Application

Software Requirements

- Node.js (version 16.x or higher)
- npm or yarn package manager
- Modern web browser (Google Chrome, Mozilla Firefox, Microsoft Edge)

Hardware Requirements

- **Processor:** Minimum Dual-Core CPU
- **RAM:** At least 4 GB (8 GB recommended)
- **Storage:** Minimum 500 MB free space for application files and dependencies

4. IMPLEMENTATION

The implementation of **CodeSync** is carried out in a modular and phased manner to ensure scalability, real-time performance, and seamless collaboration. The platform is designed to support multiple users working together in a shared coding environment while maintaining synchronization, security, and usability. Each module is independently developed and then integrated into a unified collaborative workspace.

1. User & Collaboration Module

- **Room Creation and Joining:**

- Users can create or join collaboration rooms using a unique Room ID. This functionality is implemented using Node.js and Socket.io, enabling multiple participants to connect to the same workspace in real time.

- **Authentication:**

Lightweight session management ensures secure access to rooms without requiring complex sign-up processes. Socket-based authentication validates users upon joining a session.

- **Identification:**

Users are assigned roles such as host or participant, allowing controlled access to editing, execution, and collaboration features.

2. Real-Time Code Editor Module

- **Editing:**

The core code editor is built using React.js and supports real-time multi-user editing. Socket.io is used to synchronize code changes instantly across all connected clients.

- **Highlighting:**

Live cursor positions and text selections are broadcast to all users, enabling clear visibility of collaborators' actions.

- **Multi-Languages:**

The editor supports multiple programming languages with syntax highlighting and formatting to accommodate diverse development needs.

3. Code Execution & Live Preview Module

- **Terminal:**

A shared terminal allows users to execute code and view output collectively. Execution requests are handled by the backend and results are streamed back to all participants.

- **Preview:**

Frontend changes are rendered instantly using integrated preview components, enabling rapid UI testing and debugging.

4. Communication & Documentation Module

- **Notepad:**

A real-time collaborative notepad allows users to write notes, ideas, and documentation using rich text and Markdown support.

- **In-Room:**

Basic messaging and interaction features are integrated to facilitate discussion without leaving the platform.

5. Version Control Integration

- **GitHub:**

CodeSync integrates Git and GitHub to manage version control operations such as commit, push, pull, branching, and merging directly from the platform.

- **Change:**

Code changes are tracked to maintain consistency and allow rollback if required

6. System Integration

- **Frontend–Backend:**

REST APIs built using Express.js handle room management, code storage, and version control operations. Socket.io manages real-time synchronization.

- **Database:**

MongoDB is used to store room data, code snapshots, user activity logs, and shared notes.

- **Synchronization:**

All collaborative actions—code edits, cursor movements, notes, and terminal outputs—are synchronized in real time to ensure a smooth user experience.

7. Deployment

- **Frontend:**

Deployed on Vercel to ensure fast loading, responsiveness, and scalability.

- **Backend: &**

Backend services are hosted on cloud platforms such as Render, with MongoDB providing reliable data storage.

- **Version:**

GitHub is used for source code management, enabling continuous integration and smooth deployment updates.

Through this modular and integrated implementation, **CodeSync** delivers a powerful real-time collaborative coding platform that enhances productivity, learning, and teamwork. The system is scalable and extensible, allowing future enhancements such as video conferencing, AI-assisted code suggestions, and automated code reviews.

5. APPENDICES

[illegible]

6. Data Base Schema

USER

PK | user_id (UUID)

| username (VARCHAR, UNIQUE)

| email (VARCHAR, UNIQUE)

| password_hash (VARCHAR)

| profile_pic (TEXT)

| bio (TEXT)

| created_at (TIMESTAMP)

| last_login (TIMESTAMP)

| account_status (ENUM: 'active', 'suspended', 'deleted')

PROJECT

PK | project_id (UUID)

| name (VARCHAR)

| description (TEXT)

| FK | owner_id (UUID) -> USER.user_id

| created_at (TIMESTAMP)

| updated_at (TIMESTAMP)

| visibility (ENUM: 'public', 'private', 'unlisted')

| repo_url (VARCHAR)

| default_branch (VARCHAR)

PROJECT_MEMBER

PK | project_member_id (UUID)

FK | project_id (UUID) -> PROJECT.project_id

FK | user_id (UUID) -> USER.user_id

| role (ENUM: 'owner', 'admin', 'contributor', 'viewer')

| joined_at (TIMESTAMP)

| last_active (TIMESTAMP)

CODE_FILE

PK | file_id (UUID)

FK | project_id (UUID) -> PROJECT.project_id

| file_path (VARCHAR)

| content (TEXT)

| created_at (TIMESTAMP)

| updated_at (TIMESTAMP)

FK | last_updated_by (UUID) -> USER.user_id BRANCH

PK | branch_id (UUID)

FK | project_id (UUID) -> PROJECT.project_id

| name (VARCHAR)

| is_default (BOOLEAN)

| created_at (TIMESTAMP)

FK | created_by (UUID) -> USER.user_id

COMMIT

PK | commit_id (UUID)

FK | project_id (UUID) -> PROJECT.project_id

FK | branch_id (UUID) -> BRANCH.branch_id

FK | author_id (UUID) -> USER.user_id

| hash (VARCHAR)

| message (TEXT)

| timestamp (TIMESTAMP)

| changes (JSON) // Stores file changes in this commit

ISSUE

PK | issue_id (UUID)

FK | project_id (UUID) -> PROJECT.project_id

FK | created_by (UUID) -> USER.user_id

FK | assigned_to (UUID) -> USER.user_id (nullable)

| title (VARCHAR)

| description (TEXT)

| status (ENUM: 'open', 'in_progress', 'closed', 'reopened')

| priority (ENUM: 'low', 'medium', 'high', 'critical')

| created_at (TIMESTAMP)

| updated_at (TIMESTAMP)

| closed_at (TIMESTAMP, nullable)

COMMENT

PK | comment_id (UUID)

FK | issue_id (UUID) -> ISSUE.issue_id (nullable)

FK | commit_id (UUID) -> COMMIT.commit_id (nullable)

FK | user_id (UUID) -> USER.user_id

| content (TEXT)

| created_at (TIMESTAMP)

| updated_at (TIMESTAMP)

PULL_REQUEST

PK | pr_id (UUID)

FK | project_id (UUID) -> PROJECT.project_id

FK | author_id (UUID) -> USER.user_id

FK | target_branch_id (UUID) -> BRANCH.branch_id

FK | source_branch_id (UUID) -> BRANCH.branch_id

| title (VARCHAR)

| description (TEXT)

| status (ENUM: 'open', 'merged', 'closed', 'draft')

| created_at (TIMESTAMP)

| updated_at (TIMESTAMP)

| merged_at (TIMESTAMP, nullable)

REVIEW

PK | review_id (UUID)

FK | pr_id (UUID) -> PULL_REQUEST.pr_id

FK | reviewer_id (UUID) -> USER.user_id

| status (ENUM: 'approved', 'changes_requested', 'commented')

| comment (TEXT, nullable)

| submitted_at (TIMESTAMP)

CHAT_ROOM

PK | room_id (UUID)

FK | project_id (UUID) -> PROJECT.project_id

| name (VARCHAR)

| is_private (BOOLEAN)

| created_at (TIMESTAMP)

CHAT_MEMBER

PK | chat_member_id (UUID)

FK | room_id (UUID) -> CHAT_ROOM.room_id

FK | user_id (UUID) -> USER.user_id

| joined_at (TIMESTAMP)

MESSAGE

PK | message_id (UUID)

FK | room_id (UUID) -> CHAT_ROOM.room_id

FK | sender_id (UUID) -> USER.user_id

| content (TEXT)

| sent_at (TIMESTAMP)

| edited_at (TIMESTAMP, nullable)

❖ Mongo Db

```
js

const mongoose = require('mongoose');

const ImageSchema = new mongoose.Schema({
  name: String,
  img: {
    data: Buffer,
    contentType: String
  }
});

const ImageModel = mongoose.model('Image', ImageSchema);

// To save an image
const fs = require('fs');

fs.readFile('path/to/image.jpg', (err, data) => {
  if (err) throw err;

  const newImage = new ImageModel({
    name: 'My Image',
    img: {
      data: data,
      contentType: 'image/jpeg'
    }
  });

  newImage.save()
    .then(() => console.log('Image saved successfully'))
    .catch(console.error);
});
```

js

```
const { MongoClient, GridFSBucket } = require('mongodb');
const fs = require('fs');

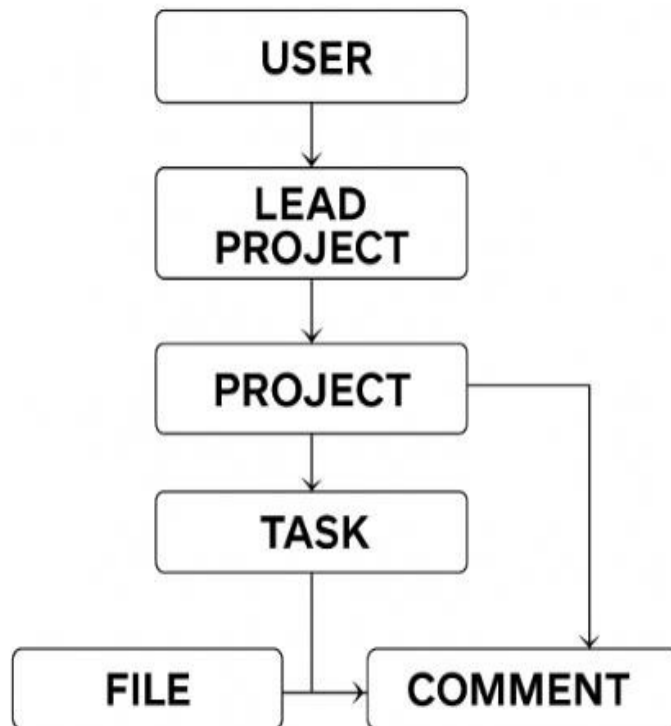
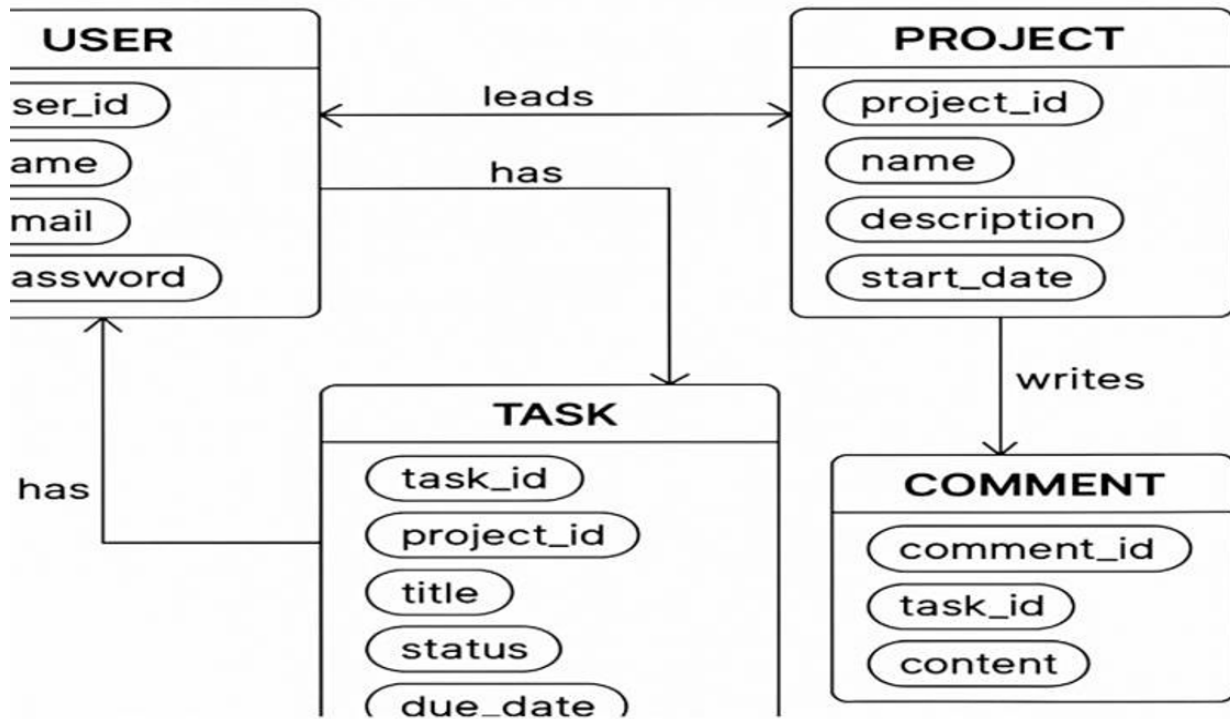
async function uploadImage() {
  const client = new MongoClient('mongodb://localhost:27017');
  await client.connect();
  const db = client.db('yourdb');

  const bucket = new GridFSBucket(db, {
    bucketName: 'images'
  });

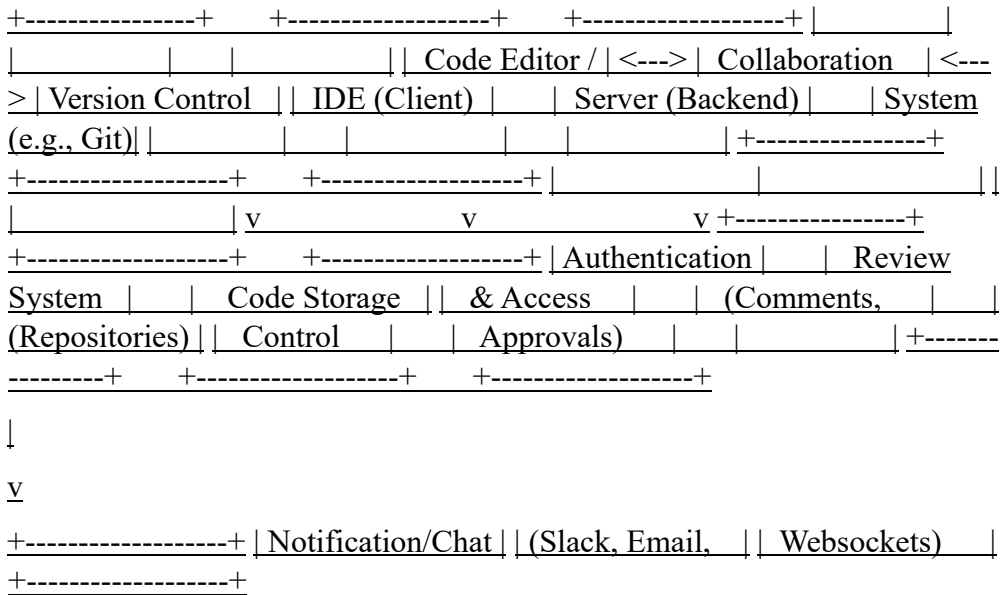
  const uploadStream = bucket.openUploadStream('myImage.jpg');
  fs.createReadStream('path/to/myImage.jpg').pipe(uploadStream)
    .on('error', (error) => console.error(error))
    .on('finish', () => {
      console.log('Image uploaded successfully');
      client.close();
    });
}

uploadImage();
```

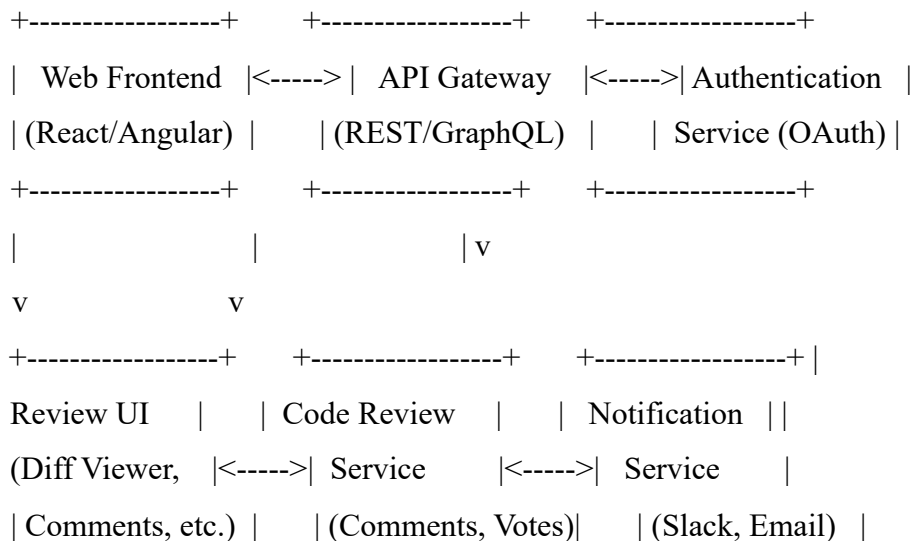
7. Entity Relationship Diagram

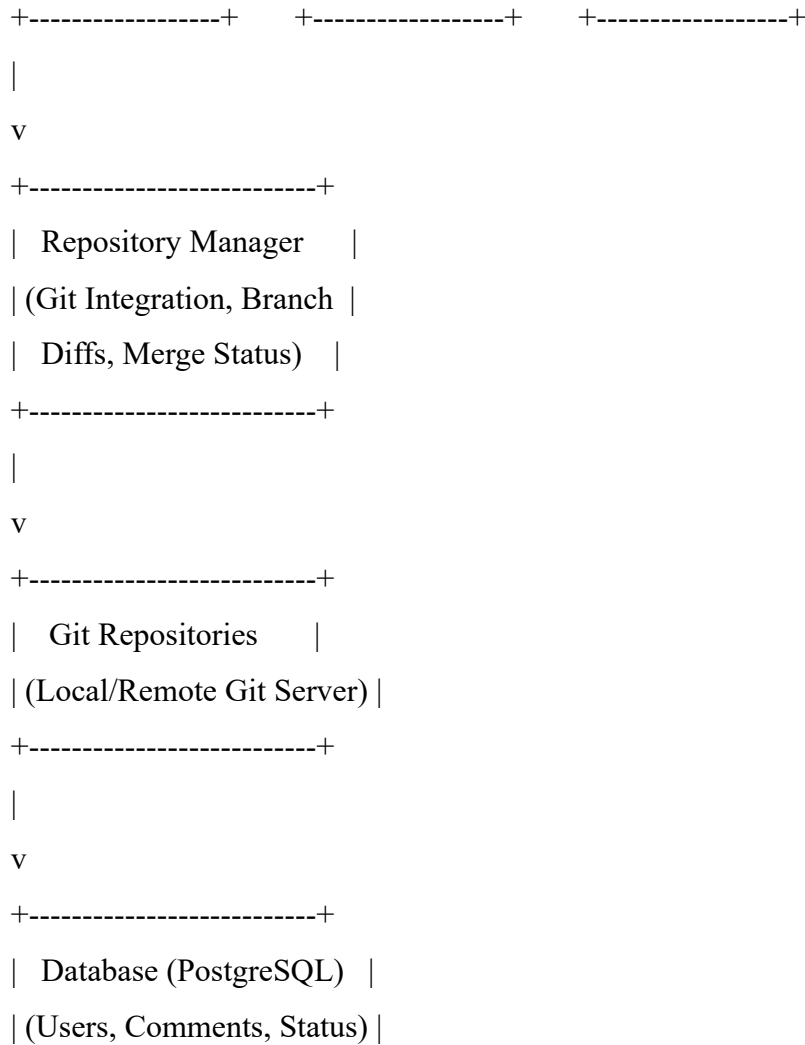


High Level Diagram



Low level Diagram





Program's Structure Analyzing

1. Lexical Analysis

Tokenize source code into variables, keywords, etc.

2. Parsing (Syntax Analysis)

Build abstract syntax trees (ASTs) from source code.

3. Tools: ANTLR, Yacc, Bison

Semantic Analysis

4. Check for type correctness, scope resolution, etc.

Dependency Analysis

5. Examine module/function/class relationships.

Generate call graphs or module dependency diagrams.

6. Control & Data Flow Analysis

Identify loops, branches, and variable usage.

7. Tools: LLVM, CodeQL, Ghidra

8. Limitations

1. Real-Time Collaboration Complexity:

- **Latency & Sync Issues:** Keeping code edits perfectly synchronized in real-time across multiple users can be tricky. Network delays and conflicts might cause inconsistencies. Conflict Resolution: Handling simultaneous edits to the same line or section of code without losing data requires advanced algorithms (like Operational Transformation or CRDTs), which are complex to implement.

2. Scalability:

- **Server Load:** As the number of users grows, real-time updates and database operations increase exponentially.

Resource Intensive: Real-time syncing, multiple websocket connections, and frequent database writes can

- **Scaling Database:** Keeping the database consistent and responsive with many simultaneous users can be challenging.

3. Security Concerns

- **Data Privacy:** Code and user data must be protected from unauthorized access.

Injection Attacks: User input (code snippets, comments) could introduce vulnerabilities like XSS or SQL injection

- **Authentication & Authorization:** Properly managing who can see/edit what is complex.

❖ **Benefits of Program Structure Analyzing and GUI Constructing**

Improved Code Understanding: Helps developers visualize the architecture (e.g., AST, control flow). Aids in debugging, refactoring, and onboarding new team members.

Faster Development: Automating GUI construction reduces time to build interfaces. Visual representations accelerate decision-making.

Enhanced Code Quality: Structural analysis can detect bugs, dead code, and design issues early. Encourages clean, maintainable, and well-structured codebases.

Better UI/UX Design: A systematic GUI builder ensures consistency and usability. Easier to implement design systems and accessibility features.

Automation of Repetitive Tasks: Saves time on manual diagramming and GUI layouting. Automatically generates graphs, trees, and components from source code.

Education and Training: Ideal for teaching compiler design, software architecture, and UI development. Students can visualize how code translates into logic and interface.

Scalability: Modular structure supports larger projects and team collaboration. GUI modules can be reused across applications.

Documentation Generation: Visual artifacts can double as documentation. Great for audits, code reviews, and architecture compliance.

9. Conclusion

Building Collab Hub, a real-time collaborative coding platform, offers an exciting opportunity to enhance teamwork and productivity among developers. By integrating features like live code editing, version control, role-based permissions, and seamless communication, CodeSynccan significantly simplify how teams work together on software projects.

However, the project comes with challenges including handling real-time synchronization, ensuring security, managing scalability, and addressing device performance variations. Careful planning of the technology stack, infrastructure, and feature set is essential to create a robust, userfriendly, and scalable platform. Starting with a focused minimum viable product (MVP) and gradually expanding features based on user feedback will help manage complexity and deliver value efficiently. With the right tools, architecture, and ongoing iteration, CodeSynccan become a powerful environment for collaborative software development.

10. Future Scope

- 1)Advanced Conflict Resolution
- 2)Implement sophisticated algorithms like CRDTs (Conflict-free Replicated Data Types) to improve real-time conflict handling and allow offline edits with seamless sync.
- 3)AI-Powered Code Assistance
- 4)Integrate AI tools for auto-completion, error detection, and code suggestions to boost productivity.
- 5)Support AI-driven code review and refactoring recommendations.
- 6)Multi-Language & Framework Support 7)Expand syntax highlighting, linting, and debugging tools to support a wider range of programming languages and frameworks.
- 8)Enhanced Collaboration Features
- 9)Add video/audio calls and screen sharing inside the platform.
- 10)Implement task management and sprint planning tools for agile workflows.

11. Bibliography/References

1. React: <https://reactjs.org/docs/getting-started.html>
2. Tailwind CSS: <https://tailwindcss.com/docs>
3. React Router: <https://reactrouter.com/en/main>
4. Material-UI: <https://mui.com/material-ui/getting-started/>
5. Node.js: <https://nodejs.org/en/docs/>
6. Express.js: <https://expressjs.com>
7. Socket.IO: <https://socket.io/docs/v4/>
8. Firebase Auth: <https://firebase.google.com/docs/auth>
9. JWT: <https://jwt.io/>
10. MongoDB: <https://docs.mongodb.com/manual/>
11. Mongoose: <https://mongoosejs.com/docs/guide.html>